

RAPPORT DE STAGE ET DOCUMENTATION TECHNIQUE DÉTAILLÉE

Projet : Agent IA RH (RecrutIA) — Module 1 : Ingestion Intelligente de CV
Entreprise d'Accueil : ARTIWEB (Fès, Maroc — Partenaire Google)

I. PRESENTATION DE L'ENTREPRISE ARTIWEB & FICHE PROJET

ArtiWeb est une agence de communication et marketing digital basée à Fès (<https://artiweb.ma/>). Partenaire Google certifié, elle propose des expertises en stratégie digitale, développement web/mobile, SEO/SEM, Social Ads et solutions E-commerce/Marketplace. Contacts : contact@artiweb.ma | (+212) 664-017447.

Problématique du projet RecrutIA : Le tri manuel des candidatures par les RH est long, coûteux, sujet à la fatigue et aux biais. L'objectif est d'automatiser l'analyse et la pré-sélection des CV grâce à un système intelligent à 3 couches (Ingestion → IA Agentique → Supervision Humaine).

II. ARCHITECTURE TECHNIQUE DU MODULE D'INGESTION

La **Couche d'Ingestion** a été construite sous la forme d'un package Python modulaire et indépendant (développé dans `agent_ai/ingestion/`) :

```
agent_ai/
├── ingestion/
│   ├── __init__.py      # Export officiel de la fonction traiter_cv()
│   ├── extractor.py     # Algorithme d'extraction cascade (PDF / DOCX / OCR Tesseract)
│   ├── cleaner.py      # Nettoyage, normalisation Unicode NFC, suppression du bruit
│   ├── parser.py       # Parsing NLP : compétences, durées d'expérience, diplômes
│   └── traiter_cv.py    # Orchestrateur central du pipeline de traitement
├── data/
│   └── competences_ref.json # Dictionnaire JSON de référence (~80 compétences)
├── mes_cv/             # Répertoire de test pour les CV réels
├── tests/
│   └── test_ingestion.py # Suite complète de 21 tests unitaires et d'intégration
├── run.py / demo_test.py # Scripts de démonstration exécutables
└── requirements.txt    # Dépendances (pdfplumber, PyMuPDF, python-docx, pytesseract, Pillow)
```

III. MÉTHODE DE FONCTIONNEMENT DÉTAILLÉE (LE PIPELINE)

Le traitement d'un CV se déroule selon un pipeline strict en 4 étapes orchestré par `traiter_cv.py` :

Étape 1 : Extraction du Texte en Cascade (`extractor.py`)

Pour garantir de lire 100% des fichiers reçus, l'extracteur applique une stratégie d'escalade :

- Fichiers .DOCX (Word)** : Utilisation de `python-docx` pour parcourir tous les paragraphes et les cellules de tableaux.
- Fichiers .PDF Textuels (Niveau 1)** : Utilisation de `pdfplumber` qui extrait le texte natif tout en conservant la disposition.
- Fichiers .PDF Complexes (Niveau 2)** : Si `pdfplumber` donne peu de texte, recours immédiat à `PyMuPDF (fitz)`.
- Fichiers .PDF Scannés / Images (Niveau 3 - Fallback OCR)** : Si le texte extrait fait < 80 caractères, `PyMuPDF` convertit chaque page en une image HD (300 DPI) et `pytesseract` (moteur OCR Tesseract) analyse les pixels pour restituer le texte.

Étape 2 : Nettoyage & Normalisation (`cleaner.py`)

Le texte extrait brut contient souvent du bruit. Le nettoyeur applique les corrections suivantes :

- Normalisation Unicode NFC** : Standardisation des encodages et conversion des ligatures PDF (ex: '■' → 'fi').
- Suppression du bruit visuel** : Élimination des symboles graphiques (•, ★, ■, ✓) via Regex.
- Correction des mots coupés** : Fusion des mots scindés par un tiret en fin de ligne (ex: 'Déve\nloppeur' → 'Développeur').
- Suppression des en-têtes/pieds de page** : Élimination des mentions répétitives type 'Page 1 sur 2'.

Étape 3 : Parsing & Extraction NLP d'Informations (`parser.py`)

Le texte propre est analysé pour extraire 3 entités clés :

- **1. Compétences** : Comparaison insensible à la casse avec le fichier competences_ref.json. Scan d'abord dans les sections dédiées ('Compétences', 'Stack', 'Outils') puis scan global.
- **2. Expérience (Années)** : Repérage des mentions explicites ('X ans d'expérience') via Regex. En l'absence de mention explicite, extraction de toutes les plages d'années (ex: 2019-2023), fusion des périodes chevauchantes et calcul du cumul réel d'années.
- **3. Formation** : Application d'une hiérarchie stricte (Doctorat/PhD > Master/Ingénieur > Licence/Bachelor > BTS/DUT > Bac). La fonction renvoie le niveau le plus élevé détecté.

Étape 4 : Assemblage du Résultat JSON (traiter_cv.py)

Le pipeline assemble les données extraites et mesure la durée d'exécution. Il génère un dictionnaire structuré au format JSON prêt pour l'IA.

IV. EXPLICATION DÉTAILLÉE DES TESTS AUTOMATISÉS (21 TESTS)

Le fichier tests/test_ingestion.py contient **21 tests automatiques** exécutés via pytest :

1. Suite Unitaires Nettoyeur (TestCleaner - 5 tests)

- test_suppression_caracteres_parasites : Vérifie le retrait des symboles (•, ★).
- test_ligatures_pdf : Vérifie la conversion de '■' en 'fi'.
- test_normalisation_unicode & test_espaces_multiples : Vérifie la propreté du texte.

2. Suite Unitaires Parseur (TestParser - 9 tests)

- test_extraction_compétences_basique & _insensible_casse : Vérifie le matching de compétences.
- test_experience_mention_explicite & _calcul_dates : Vérifie la détection directe '5 ans' et le calcul automatique à partir de dates chevauchantes.
- test_formation_priorite_plus_haut_niveau : Envoie un texte avec 'Bac' + 'Master' + 'PhD' et vérifie que la réponse est bien 'Doctorat / PhD'.

3. Suite d'Intégration Pipeline (TestIntegrationTraiterCV - 7 tests)

Ces tests créent de **vrais fichiers virtuels PDF et DOCX temporaires sur le disque** et les font passer par tout le pipeline :

- test_cv_pdf_bien_formate : PDF natif → Valide le succès et l'extraction complète.
- test_cv_pdf_mal_formate : PDF avec symboles et bruit → Valide la robustesse du nettoyeur.
- test_cv_docx : Document Word avec tableaux → Valide l'extraction python-docx.
- test_fichier_inexistant & _format_non_supporte : Valide la gestion d'erreurs propre sans crash.

Résultat de la suite de tests : 21/21 PASSED (100% de réussite en 0.91 seconde).

V. VALIDATION SUR LE CV RÉEL (MEKKI_FATIMA-EZAHRAE_AI_Backend.pdf)

Testé en conditions réelles avec le script analyser_mon_cv.py :

- **Fichier** : MEKKI_FATIMA-EZAHRAE_AI_Backend.pdf | **Statut** : SUCCÈS
- **Formation** : Master / Ingénieur (Détecté 'Ingénieure en Génie Informatique')
- **Expérience** : 2 an(s) | **Durée de traitement** : 0.782 seconde
- **33 Compétences extraites** : Python, Django, Flask, TensorFlow, React, Azure, Java, MySQL, Node.js, Scikit-learn, REST API, etc.

```
{
  "fichier": "MEKKI_FATIMA-EZAHRAE_AI_Backend.pdf",
  "statut": "succès",
  "formation": "Master / Ingénieur",
  "experience_annees": 2,
  "nb_compétences": 33,
  "compétences": ["Azure", "Bootstrap", "CSS", "Cloud", "Computer Vision", "Django", "Flask", "Python", "React", ..
.],
```

```
"duree_traitement": "0.782 s"  
}
```

VI. CONCLUSION

Le module d'ingestion développé pour **ArtiWeb** est complet, modulaire, ultra-rapide (< 1s) et totalement autonome. Il fournit le contrat de données JSON nécessaire à la future Couche d'IA Agentique (Couche 2).